PairCover-Q · Round 2 speaker script

Team Evric · Q-HACK India 2026 · 12 slides, about 11 minutes. Words in purple brackets are actions, not lines to read.

Part	Presenter	Covers	Slides
1	**Divya**	Problem statement	Slides 1–3
2	**Harsha**	Solution and the mathematics	Slides 4–6
3	**You**	Live demo, simulation and IBM hardware results	Slides 7–9
4	**Jayasuriya**	Real quantum vs classical computing, innovation and impact	Slides 10–12

Divya

Slide 1: Cover · about 0:30

Good morning, everyone. We are Team Evric, in the Quantum Computing and Optimization track, and our team lead is Gautham M A.

Our project is PairCover-Q: quantum-assisted ambulance allocation for optimized emergency response.

I'm Divya, and I will start with the problem. Then Harsha will explain our solution and the mathematics behind it. After that we will show you our working app, our simulation results and our results from real IBM quantum computers. Finally, Jayasuriya will compare real quantum computing with classical computing and talk about the impact.

Divya **Slide 2: Problem** · about 1:00

Today, most ambulance dispatch follows one simple rule: send the nearest free ambulance to each emergency. For one call at a time, that works.

But look at this example from our app. Two emergencies, C2 and C3, arrive together, and we have three free ambulances. A1 and A2 are backup partners: they cover the same neighbourhood for each other.

The nearest-ambulance rule sends A1 to C2 and A2 to C3. Both partners leave, and their whole area is left with no backup. In our cost model that plan costs 22.

PairCover-Q keeps A2 at home and sends A3 to C3 instead. C3 waits just one minute longer, but the area stays protected. The cost drops to 13. That is 41 percent lower, on the same two calls.

So the problem is not only who is closest. It is how to plan several calls together so that no area is left empty.

Divya **Slide 3: Why It Matters** · about 0:50

Why does this matter in India? First, policy is moving this way. The 2026 National Ambulance Services guidelines call for integrated command centres, intelligent dispatch and data-driven deployment.

Second, ambulances are scarce. The national norm is one advanced life support ambulance for every 5 lakh people. When a backup is gone, it is hard to replace.

Third, services work against fixed targets. For example, Karnataka's 108 service targets 20 minutes in cities and 30 minutes in rural areas, so every minute counts.

PairCover-Q is decision support for a 108 command centre at exactly the moment several calls arrive together. It recommends who goes where; the dispatcher still decides.

Now Harsha will explain how we solve it.

Harsha **Slide 4: Proposed Solution** · about 0:50

Thank you, Divya. Our idea is to turn dispatch into one optimisation problem for all active calls together, instead of one call at a time.

It works in five steps. One: we take the waiting calls, the free ambulances and the travel times. Two: we build a QUBO, with one yes-or-no variable for every possible ambulance-to-call pair. Each variable becomes one qubit. Three: we build a QAOA circuit in Qiskit. Four: we run it on a simulator, on IBM's noise copy of a real chip, and on real IBM quantum computers. Five: we check every answer against an exact classical solver and against random guessing.

The cost we minimise has three parts: priority-weighted travel time, a penalty if a call is left unserved, and a penalty for coverage lost when ambulances leave their area.

The key point: losing a backup pair is a product of just two yes-or-no variables. That is exactly the form a QUBO accepts, so coverage needs no extra qubits.

Harsha **Slide 5: Mathematical Model** · about 1:10

Here is the exact model in our code. The decision variable x-u-j is 1 if ambulance u goes to call j. We only create variables for useful pairs: the ambulance can arrive within 12 minutes, or it is the nearest one. That keeps the circuit small.

The objective F of x has four terms. First, call priority times travel minutes. Second, a penalty U-j for every call nobody serves. Third, the coverage an ambulance holds on its own, lost if it leaves. Fourth, the shared coverage of a backup pair, lost only if both partners leave. That last term is y-a times y-b, a product of two variables.

The constraints are simple: one call per ambulance and one ambulance per call. A quantum circuit cannot enforce constraints directly, so we move them into the cost as a penalty P.

This gives our QUBO, H of x, with linear terms h-k and pair terms J-k-l. We choose P as the largest possible gain plus one. That is the smallest value that provably makes every minimum a valid plan. We also measured that this tight penalty works better for QAOA than a large one.

Finally, we substitute x equals one minus Z, over two. That turns the QUBO into an Ising Hamiltonian C, which is what the quantum computer runs.

Harsha **Slide 6: Why Quantum?** · about 1:00

Why quantum? Because our problem is already a QUBO, and QAOA, the Quantum Approximate Optimization Algorithm, is the standard algorithm for QUBOs on today's quantum computers.

This is our real circuit for a 4-qubit dispatch problem, drawn by Qiskit. Step one: the H gates put every possible dispatch plan into superposition at once. Step two, the cost layer: RZ and RZZ rotations, whose angles are our QUBO coefficients, write each plan's cost into its phase. There is one RZZ for every pair of coupled choices. Step three, the mixer: RX rotations let the plans interfere, so low-cost plans gain probability.

A classical optimiser, COBYLA, tunes the angles gamma and beta to minimise the expected cost. Then we measure 4,096 times and decode every bitstring back into a dispatch plan. In this circuit, qubit 2 is A3 going to call 1, which is the best plan.

So it is a hybrid: the quantum circuit explores, and classical code builds, tunes and checks. Now my teammate will show it working, and the results.

You **Slide 7: Working Prototype** · about 1:45 (live demo)

Thank you, Harsha. Everything we have described runs in a live web app. Let me show you.

[Click Home, then 'See the decision in one click'.] This runs the nearest-ambulance rule and our exact optimiser on the example Divya showed. You can see the cost go from 22 to 13, and A2 stays home.

[Open Quantum, then Classical vs Quantum, '4 qubits, depth 2'.] This page solves the same problem four ways. The nearest rule and the exact search run live in under a millisecond. QAOA on the simulator runs live too. And these rows are the same circuit run on IBM's real 156-qubit chips. Each row shows the answer, how often it hit the best plan, the gate count and the time.

[Click '6 qubits, depth 2'.] At 6 qubits, look: the nearest rule now picks a worse plan than the exact solver, because it ignores coverage. And the real chip is only around random guessing here. We will come back to that.

[If time allows: Try it, then Create your own emergency.] You can also place your own emergencies on a map and every solver runs live.

You **Slide 8: Results: real IBM hardware** · about 1:00

Our mentor told us that real hardware results would be our biggest advantage, so we ran them. In total, 9 jobs on two IBM Heron chips with 156 qubits each: ibm_fez and ibm_marrakesh.

This chart shows the strongest problem at each size, from 4 to 10 qubits. Above the dotted line means the chip finds the best dispatch more often than random guessing among valid plans. The dashed line is the noiseless simulator, and the solid line is the real chip.

At 4 qubits the real chip finds the best plan 2.8 times more often than guessing, and the best plan was its most frequent answer in both runs. Even at 10 qubits, with 284 two-qubit gates, it is still 2.7 times better. At 7 and 8 qubits, noise pushes it slightly below guessing.

The noiseless line stays above guessing at every size. So the limit is hardware noise, not our algorithm. We also checked that this is not just a noise bias: the chip beats a noise-only baseline on every problem.

You **Slide 9: Results: simulated city** · about 0:50

One example is not enough, so we also tested the method over time. We built a simulated city, 24 by 24 kilometres, with 7 ambulances and 6 stations, and ran it for 7 days, 30 times at each call load. Every policy saw exactly the same calls.

At 5 calls per hour, mean response time fell by 2.15 minutes compared with nearest-available, with a 95 percent confidence interval of 1.63 to 2.67 minutes. Late arrivals fell by 1.8 points. And uncovered demand fell at all 4 loads we tested.

To be honest about it: most of the response-time gain comes from planning simultaneous calls together, and the coverage term lowers uncovered demand. The city is synthetic; a real pilot would use 108 call logs.

Now Jayasuriya will compare real quantum computing with classical computing.

Jayasuriya **Slide 10: Quantum vs classical** · about 1:15

Thank you. Let me be direct about quantum versus classical, because this is where many projects overclaim.

On speed, classical wins today. The exact classical search answers in under a millisecond. The noiseless quantum simulator takes one to four milliseconds. On real IBM hardware, a job took 11 seconds with an empty queue and up to 20 minutes with a busy queue, even though the circuits ran on the chip for only 3 to 5 seconds.

How do we run on the real chip? We transpile our circuit with Qiskit to the chip's native gates. The 4-qubit circuit becomes 33 two-qubit gates on four physical qubits. We submit it through Qiskit Runtime with 4,096 shots, and every job carries calibration circuits for readout errors.

We tried error mitigation. Readout correction added about one point. Dynamical decoupling and twirling did not help. We also saw the same circuit, on the same qubits, go from 28 to 48 percent within two hours, so the chip drifts. That tells us the real limit is two-qubit gate noise.

So we claim no quantum advantage. What we do claim is a complete quantum pipeline, from an emergency to a real IBM chip and back to a correct dispatch decision, with an honest, measured boundary of where it helps today.

Jayasuriya **Slide 11: Innovation, impact and roadmap** · about 1:00

What is new in our work? Four things. One: coverage is a pairwise term, so coverage-aware dispatch needs no extra qubits. Two: a penalty that is provably safe and tight. Three: small circuits by design, because only useful pairs get a qubit. Most real decisions need about 2 qubits, and 8 to 12 only when calls coincide. Four: honest validation on two real IBM chips, with repeats, mitigation and baselines.

The impact: faster response by planning calls together, balanced coverage so no area is left empty, and a pipeline that is ready to grow with quantum hardware, because more simultaneous calls means more qubits.

Our roadmap attacks exactly the limit we measured. The real hardware run is done. Next is zero-noise extrapolation for gate noise, an XY mixer to keep samples valid, runs on more chips and more days, and then a pilot with real 108 call logs from Tamil Nadu's research dataset.

Jayasuriya **Slide 12: References and thank you** · about 0:20

These are our sources, and all our code, reports and hardware results are public on GitHub.

To close: we took ambulance dispatch from a QUBO to a real IBM quantum computer and back to a correct decision, and we measured honestly where quantum helps today. Thank you. We are happy to take your questions.

Questions and who answers them

“Is this faster than classical?” Jayasuriya

No. Exact search takes under a millisecond; a hardware job took 11 seconds to 20 minutes. We claim a validated pipeline and a measured boundary, not speed.

“Why not a classical solver like OR-Tools or an ILP?” Jayasuriya

For one batch, a classical solver wins today, and we use exact classical search as our answer key. Our point is that the problem is natively a QUBO, and we measured how far real quantum hardware gets on it.

“Did you really run on IBM hardware?” You

Yes: ibm_fez and ibm_marrakesh, IBM Heron, 156 qubits, 9 jobs, 4,096 shots per circuit. Job IDs and raw counts are in reports/ibm_hardware in our repository.

“Is the hardware result just noise?” You

We built a noise-only baseline from the chip's own per-qubit 1-rates. The chip beats it on every problem, 1.6 to 6.9 times.

“Did you try error mitigation?” Jayasuriya

Yes. Readout correction added about 1 point; dynamical decoupling with twirling did not help. The limit is two-qubit gate noise, so zero-noise extrapolation is next.

“Why is the 6-qubit result worse than the 10-qubit one?” You

Noise keeps 24 to 74 percent of the noiseless gain. The 10-qubit problem starts with a large gain, 7.2 times, so it is still 2.7 times after noise; smaller gains get lost.

“How many qubits does a real decision need?” Harsha

One per eligible ambulance-call pair. Most real decisions need about 2; 8 to 12 only when several calls arrive together.

“Why set the penalty just above the bound?” Harsha

It is the smallest value that provably keeps every minimum valid. Bigger penalties flatten the differences between plans: 2.16 times guessing at P against 0.62 times at 4P.

“Is the data real?” Divya

No, the city and calls are synthetic and labelled that way. Tamil Nadu's 108 research dataset has the fields for a real pilot.

“Who would use this?” Divya

A dispatcher in a 108 command centre, as decision support when several calls arrive together. The dispatcher decides.

Rule for every answer: say the number, then stop. If you don't know: “We have not tested that yet; it is on our roadmap.”